

Route: Esposizione Esterna delle Applicazioni

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Spiegare cos'è una **Route** in OpenShift e come differisce da un **Ingress** Kubernetes standard
- Descrivere il ruolo dell'HAProxy Router come ingress controller nativo di OCP
- Distinguere i tre tipi di terminazione TLS (edge, passthrough, re-encrypt) e scegliere quello appropriato
- Creare Route HTTP e HTTPS usando **oc expose** e **oc create route**
- Configurare annotazioni Route per timeout, HSTS e sessioni persistenti su ARO

Teoria e Concetti Chiave

Route vs Ingress Kubernetes

In Kubernetes standard si usa la risorsa **Ingress** per esporre applicazioni HTTP/HTTPS verso l'esterno. OpenShift introduce la risorsa **Route** come estensione CRD che offre:

Caratteristica	Kubernetes Ingress	OpenShift Route
API Group	networking.k8s.io/v1	route.openshift.io/v1
Ingress controller	Dipende dal deployment (nginx, traefik...)	HAProxy (incluso in OCP)
TLS termination	Configurazione via annotation	Campo nativo spec.tls.termination
Passthrough TLS	Via annotation (non standard)	Campo nativo
Wildcard hostname	Limitato	Supportato
Annotazioni specifiche	Dipende dal controller	Standardizzate per HAProxy
Pesi tra backend	Non standard	spec.alternateBackends con peso

 **Suggerimento:** OpenShift supporta anche **Ingress** Kubernetes standard, che viene convertito automaticamente in **Route** dall'Ingress Operator. Tuttavia, per sfruttare tutte le funzionalità OCP, usare **Route** nativamente.

HAProxy Router

Il **Router** di OpenShift è implementato come Pod HAProxy che gira nel namespace `openshift-ingress`. Ogni Route viene tradotta in una configurazione HAProxy che definisce frontend, backend e regole TLS.

Flusso di una richiesta:

```
Client → Azure LB (IP *.apps) → HAProxy Router Pod → Service → Pod applicativo
```

Componenti chiave:

- **IngressController** (CRD): configura il router (numero di repliche, dominio wildcard, TLS predefinito)
- **Router Pod** (`router-default-xxx`): implementa il bilanciamento HAProxy
- **Route**: oggetto che dichiara il mapping hostname → Service

```
# Namespace del router
oc get pods -n openshift-ingress

# IngressController
oc get ingresscontroller default -n openshift-ingress-operator -o yaml
```

Dominio Wildcard di ARO

Su ARO, il cluster viene configurato con un dominio wildcard predefinito:

```
*.apps.<cluster-name>.<azure-region>.aroapp.io
```

Ogni Route senza hostname esplicito ottiene automaticamente un hostname nella forma:

```
<service-name>-<namespace>.apps.<cluster-name>.<azure-region>.aroapp.io
```

Tipi di Terminazione TLS

Edge (Più comune)

```
Client → HTTPS → HAProxy (termina TLS) → HTTP → Service → Pod
```

- Il certificato TLS risiede nell'HAProxy (o nella Route)
- Il traffico **dal router al Pod è HTTP non cifrato** (sicuro se il traffico rimane nel cluster)
- Indicato quando l'applicazione non gestisce TLS nativamente

Passthrough

```
Client → HTTPS → HAProxy (passa il TCP stream) → HTTPS → Service → Pod
```

- HAProxy non decifra il traffico, usa **SNI** per il routing
- Il certificato TLS risiede nel Pod applicativo
- Il router non può ispezionare il contenuto HTTP (no cookie-based affinity, no header manipulation)
- Indicato per applicazioni che richiedono mutual TLS (mTLS) end-to-end

Re-encrypt

```
Client → HTTPS → HAProxy (termina TLS, ri-cifra) → HTTPS → Service → Pod
```

- Il router decifra con il certificato della Route, poi ri-cifra verso il Pod con un diverso certificato
- Necessita di un certificato nel Pod (es. generato da `oc create secret tls`)
- Indicato per applicazioni che richiedono cifratura end-to-end con certificato interno

Route HTTP (Non Sicura)

Una Route senza campo `spec.tls` espone il servizio su HTTP puro (porta 80):

```
spec:
  host: webapp.apps.mycluster.eastus.aroapp.io
  to:
    kind: Service
    name: webapp
  port:
    targetPort: http
```

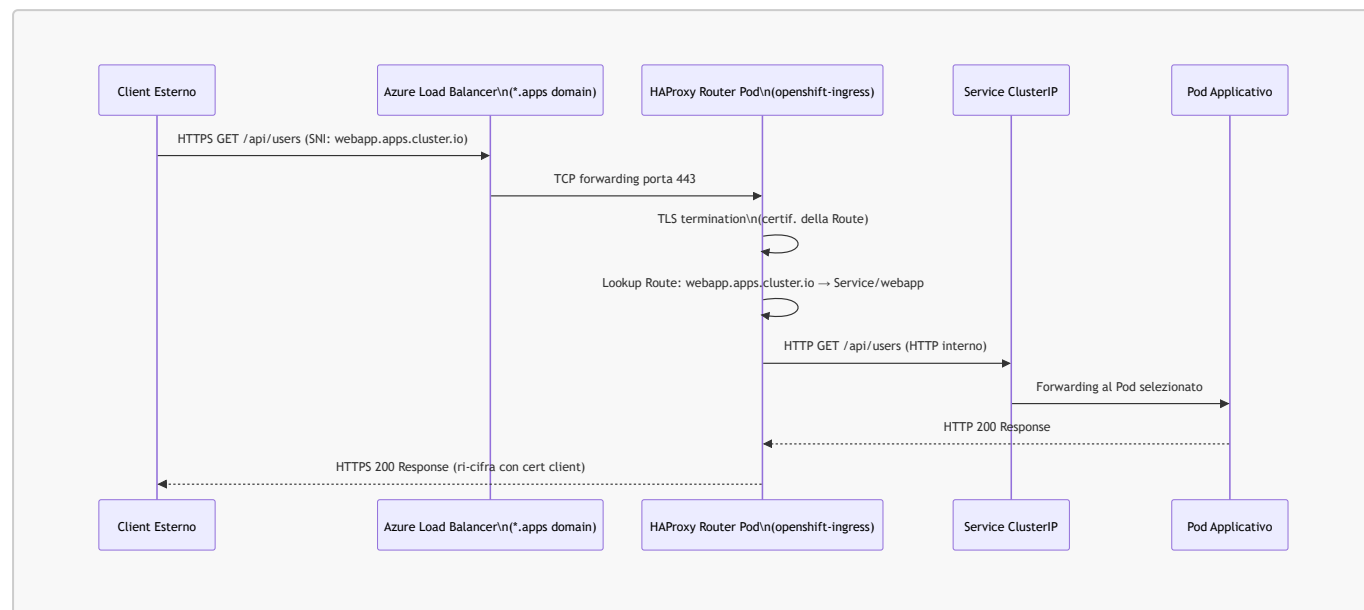
 **Attenzione:** Per produzione, usare sempre almeno la terminazione TLS edge. Route HTTP possono essere abilitate solo per servizi interni di testing o redirect HTTP→HTTPS.

Annotazioni Route Utili

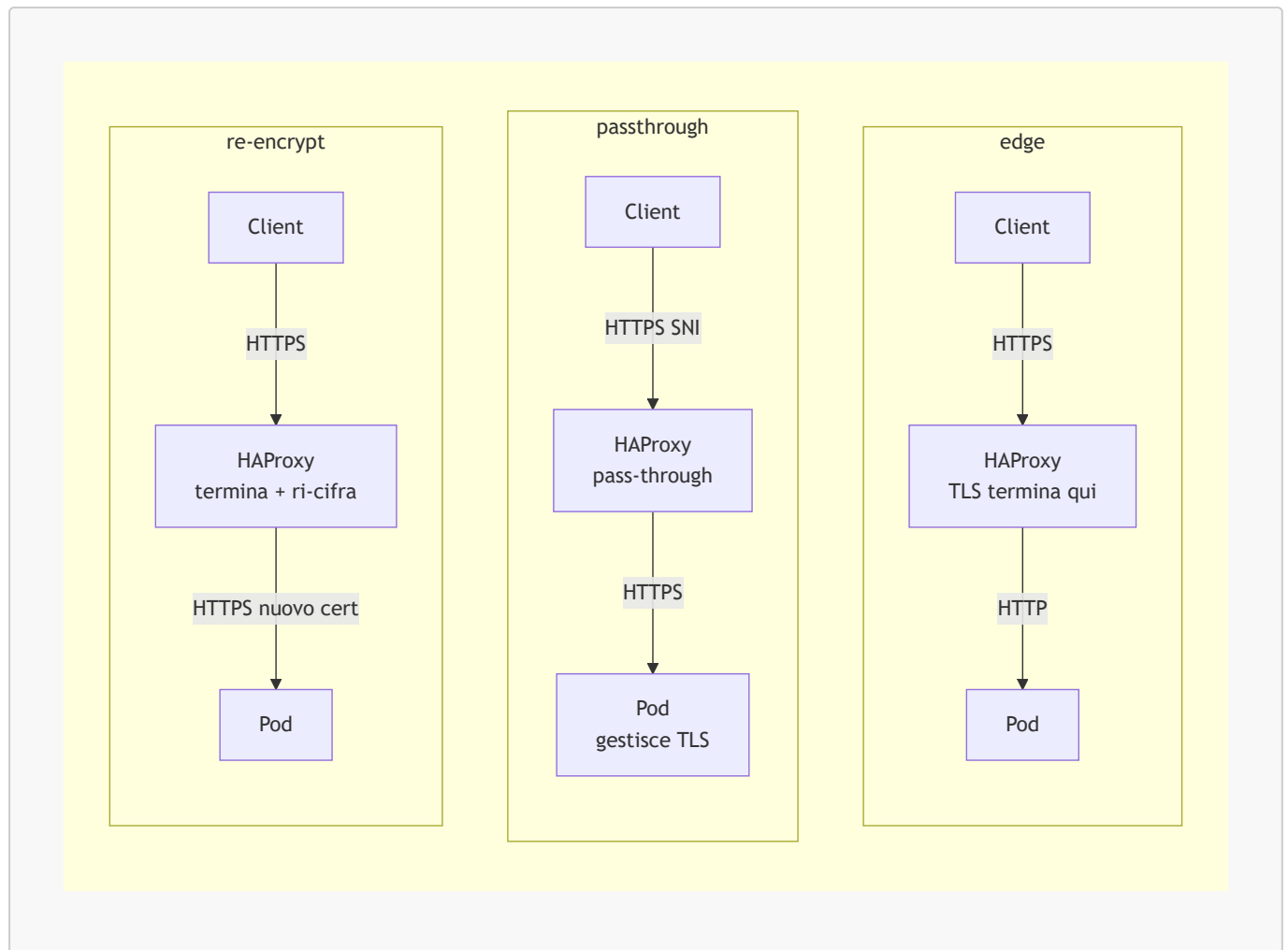
Annotazione	Descrizione	Valore esempio
<code>haproxy.router.openshift.io/timeout</code>	Timeout connessione	<code>"60s"</code>
<code>haproxy.router.openshift.io/balance</code>	Algoritmo LB	<code>"leastconn", "roundrobin"</code>
<code>haproxy.router.openshift.io/rate-limit-connections</code>	Rate limiting	<code>"true"</code>
<code>haproxy.router.openshift.io/rate-limit-connections.rate-http</code>	Req/s per IP	<code>"100"</code>
<code>haproxy.router.openshift.io/hsts_header</code>	HSTS header	<code>"max-age=31536000;includeSubDomains"</code>
<code>haproxy.router.openshift.io/cookie_name</code>	Cookie per sessione	<code>"JSESSIONID"</code>
<code>router.openshift.io/cookie-same-site</code>	SameSite cookie	<code>"Lax"</code>

Diagrammi

Flusso Richiesta HTTPS Edge



Confronto Tipi di Terminazione TLS



Comandi Pratici con Output Atteso

```
# Esporre un Service come Route HTTP (senza TLS)
oc expose service webapp --name=webapp-route
```

```
# Output atteso:
# route.route.openshift.io/webapp-route exposed
```

```
# Visualizzare le Route nel namespace corrente
oc get routes
```

```
# Output atteso:
# NAME          HOST/PORT                                PATH   SERVICES
PORT  TERMINATION  WILDCARD
# webapp-route  webapp-myproject.apps.mycluster.eastus.aroapp.io  webapp
http   None          None
```

```
# Creare una Route TLS edge con hostname personalizzato
oc create route edge webapp-tls \
  --service=webapp \
  --hostname=webapp.apps.mycluster.eastus.aroapp.io \
  --port=8080

# Output atteso:
# route.route.openshift.io/webapp-tls created
```

```
# Creare una Route TLS edge con certificato custom
oc create route edge webapp-custom-tls \
  --service=webapp \
  --hostname=webapp.example.com \
  --cert=tls.crt \
  --key=tls.key \
  --ca-cert=ca.crt

# Output atteso:
# route.route.openshift.io/webapp-custom-tls created
```

```
# Creare una Route TLS passthrough (il Pod gestisce TLS)
oc create route passthrough webapp-passthrough \
  --service=webapp-tls \
  --hostname=webapp-secure.apps.mycluster.eastus.aroapp.io

# Output atteso:
# route.route.openshift.io/webapp-passthrough created
```

```
# Creare una Route TLS re-encrypt
oc create route reencrypt webapp-reencrypt \
  --service=webapp \
  --hostname=webapp-reenc.apps.mycluster.eastus.aroapp.io \
  --dest-ca-cert=internal-ca.crt

# Output atteso:
# route.route.openshift.io/webapp-reencrypt created
```

```
# Dettaglio completo di una Route (host, TLS, annotazioni)
oc describe route webapp-tls

# Output atteso:
# Name:          webapp-tls
# Namespace:     myproject
```

```
# Host:          webapp.apps.mycluster.eastus.aroapp.io
# TLS Termination: edge
# Insecure Policy: Redirect
# Service:       webapp, weight 100 (100%)
# Port:          8080 -> 8080
# Endpoint Port: 8080
```

```
# Testare la Route HTTPS (accettando certificati self-signed)
curl -k https://webapp.apps.mycluster.eastus.aroapp.io/healthz

# Output atteso:
# {"status":"ok","version":"1.2.3"}
```

```
# Aggiungere annotazioni per timeout e HSTS su una Route esistente
oc annotate route webapp-tls \
  haproxy.router.openshift.io/timeout=120s \
  haproxy.router.openshift.io/hsts_header="max-age=31536000;includeSubDomains"

# Output atteso:
# route.route.openshift.io/webapp-tls annotated
```

Esempi YAML Completi

Route TLS Edge con Redirect HTTP→HTTPS e Annotazioni

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: webapp-edge
  namespace: myproject
  labels:
    app: webapp
  annotations:
    # ← Timeout per connessioni inattive al backend
    haproxy.router.openshift.io/timeout: "120s"
    # ← HSTS header per forzare HTTPS nel browser
    haproxy.router.openshift.io/hsts_header: "max-age=31536000;includeSubDomains;preload"
    # ← Cookie specifico per sticky session
    haproxy.router.openshift.io/cookie_name: "SESSIONID"
    # ← Algoritmo di bilanciamento
    haproxy.router.openshift.io/balance: "leastconn"
spec:
  host: webapp.apps.mycluster.eastus.aroapp.io # ← hostname pubblico della Route
  to:
    kind: Service
```



```

    name: webapp                                # ← Service di backend
    weight: 100                                # ← peso (100 = tutto il
traffico)
    port:
      targetPort: http                          # ← nome della porta nel Service
    tls:
      termination: edge                         # ← TLS termina all'HAProxy
      insecureEdgeTerminationPolicy: Redirect   # ← HTTP → HTTPS redirect
automatico
  # certificate e key: omessi → usa certificato wildcard del cluster
  # Per certificato custom:
  # certificate: |
  #   -----BEGIN CERTIFICATE-----
  #   ...
  # key: |
  #   -----BEGIN EC PRIVATE KEY-----
  #   ...
wildcardPolicy: None

```

Route TLS Re-encrypt con Certificato Interno

```

apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: webapp-reencrypt
  namespace: myproject
  labels:
    app: webapp
spec:
  host: webapp-secure.apps.mycluster.eastus.aroapp.io
  to:
    kind: Service
    name: webapp-tls                                # ← Service che espone la porta
HTTPS del Pod
    weight: 100
  port:
    targetPort: 8443                                # ← porta HTTPS del container
  tls:
    termination: reencrypt                          # ← termina TLS e ri-cifra verso
il Pod
    insecureEdgeTerminationPolicy: Redirect
    # Certificato CA interno usato per verificare il certificato del Pod
    destinationCACertificate: |
      -----BEGIN CERTIFICATE-----
      MIIBxTCCA...
      -----END CERTIFICATE-----

```

Note Specifiche per Azure ARO

Dominio ARO e Azure DNS

Su ARO, il dominio wildcard `*.apps.<cluster>.<region>.aroapp.io` è gestito da Azure DNS e punta all'IP dell'Azure Load Balancer del router. Per usare un dominio custom:

```
# 1. Ottenere l'IP del router
oc get svc router-default -n openshift-ingress \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}'
# Output: 20.56.78.90

# 2. Creare un record CNAME o A nel proprio DNS per puntare all'IP del router
# Es. *.webapp.example.com → 20.56.78.90

# 3. Creare una Route con il dominio custom
oc create route edge webapp-custom \
  --service=webapp \
  --hostname=webapp.example.com \
  --cert=tls.crt \
  --key=tls.key
```

Certificato Wildcard Custom su ARO

Per sostituire il certificato wildcard del router di default ARO con un certificato custom (es. da Let's Encrypt o CA aziendale):

```
# Creare il Secret con il certificato
oc create secret tls custom-router-cert \
  --cert=wildcard.example.com.crt \
  --key=wildcard.example.com.key \
  -n openshift-ingress

# Aggiornare il IngressController per usare il certificato custom
oc patch ingresscontroller default -n openshift-ingress-operator \
  --type=merge \
  --patch='{ "spec": { "defaultCertificate": { "name": "custom-router-cert" } } }'
```

⚠ Attenzione: La modifica del certificato del router causa un riavvio dei pod HAProxy in `openshift-ingress`. Pianificare la manutenzione di conseguenza.

IngressController su ARO

ARO crea automaticamente un **IngressController** chiamato **default** che usa il service di tipo LoadBalancer:

```
# Visualizzare la configurazione del router
oc get ingresscontroller default -n openshift-ingress-operator -o yaml
# domain: apps.<cluster>.<region>.aroapp.io
# endpointPublishingStrategy: type: LoadBalancerService

# Scalare il router (per alta disponibilità)
oc patch ingresscontroller default -n openshift-ingress-operator \
  --type=merge \
  --patch='{"spec":{"replicas":3}}'
```

Riepilogo dei Punti Chiave

- La **Route** è la risorsa nativa OCP per esporre applicazioni HTTP/HTTPS verso l'esterno, implementata dall'HAProxy Router in **openshift-ingress**
 - I tre tipi di terminazione TLS sono: **edge** (TLS termina all'HAProxy), **passthrough** (TLS end-to-end al Pod), **re-encrypt** (TLS termina + ri-cifra verso il Pod)
 - **insecureEdgeTerminationPolicy: Redirect** forza il redirect HTTP→HTTPS automatico
 - Le annotazioni HAProxy permettono di configurare timeout, bilanciamento, rate limiting e HSTS
 - Su ARO il dominio default è ***.apps.<cluster>.<region>.aroapp.io**; è possibile configurare domini custom
 - **oc expose service** crea una Route HTTP di base; **oc create route edge/passthrough/reencrypt** offre controllo completo
-

Domande di Verifica

1. Un'applicazione Java gestisce TLS con il proprio keystore. Quale tipo di terminazione Route permette connessioni HTTPS end-to-end senza che HAProxy decifri il traffico?

- A) edge
- B) re-encrypt
- C) passthrough ☒
- D) none

2. Un team vuole che le richieste HTTP alla propria Route vengano automaticamente redirezionate a HTTPS. Quale configurazione è corretta?

- A) `spec.tls.termination: redirect`
- B) `spec.tls.insecureEdgeTerminationPolicy: Redirect` ☒
- C) `spec.tls.insecureEdgeTerminationPolicy: Allow`
- D) Aggiungere l'annotazione `nginx.ingress.kubernetes.io/ssl-redirect: "true"`

3. Un'applicazione richiede sticky session tramite cookie di sessione. Quale annotazione Route è corretta?

- A) `haproxy.router.openshift.io/session-cookie: "JSESSIONID"`
- B) `haproxy.router.openshift.io/sticky: "true"`
- C) `haproxy.router.openshift.io/cookie_name: "JSESSIONID"` ☒
- D) `haproxy.router.openshift.io/balance: "source"`

4. Una Route è stata creata senza specificare `spec.host`. Quale sarà l'hostname assegnato automaticamente su ARO?

- A) `<service>.<namespace>.cluster.local`
- B) `<service>-<namespace>.apps.<cluster>.<region>.aroapp.io` ☒
- C) Nessun hostname: la Route sarà non funzionante
- D) `<namespace>.<service>.svc.cluster.local`

5. Quale comando crea una Route TLS edge per il Service `api` sulla porta 8080 con hostname custom?

- A) `oc expose service api --tls=edge --hostname=api.example.com`
- B) `oc create route edge api --service=api --hostname=api.example.com --port=8080` ☒
- C) `oc create ingress api --rule=api.example.com/*=api:8080`
- D) `oc expose service api --type=edge --cert=tls`